

Atlas AQAR

أطلس عقار

Agentic AI-Driven Real-Estate Research & Investment Intelligence System

Project Plan, Methodology & 6-Engineer Work Breakdown Structure (WBS)

DOCUMENT TYPE Project Execution Blueprint	PROGRAM DEPI Round 5	EXECUTION CYCLE 4 Weeks / 28 Days
REPOSITORY nhahub/NHA-5-072	TEAM ALLOCATION 6 Dedicated Engineers	TECHNICAL DOMAIN AI / Machine Learning

LangGraph Workflow · Multi-Agent Cognitive Loop · Deterministic Financial Math · ChromaDB Vector Store · FastAPI REST Microservice · Streamlit Dashboard · Docker Compose

Digital Egypt Pioneers Initiative (DEPI) — Project NHA-5-072 · Official Implementation Plan

Table of Contents

Comprehensive structural guide to the project plan and technical distribution.

1 Executive Summary & System Architecture — Section 1

2 Project Overview & Technical Foundation — Section 2

- **2.1** Background & Market Friction in Egyptian Real Estate — §2.1
- **2.2** Strategic Vision, Mission & SMART Objectives — §2.2
- **2.3** Project Scope Boundaries & Non-Functional Exclusions — §2.3
- **2.4** Architectural Stack & Layered Technology Matrix — §2.4
- **2.5** Empirical Performance Baseline & Acceptance Metrics — §2.5

3 Stakeholder Analysis & Governance Matrix — Section 3

- **3.1** Internal Project Stakeholders — §3.1
- **3.2** External Ecosystem Stakeholders — §3.2
- **3.3** Power / Interest 2×2 Matrix — §3.3
- **3.4** Comprehensive Stakeholder Communication Plan — §3.4
- **3.5** Strategic Engagement & Escalation Protocols — §3.5

4 Project Plan, Milestones & Risk Register — Section 4

- **4.1** Chronological Project Milestones (Sprint 1 to Sprint 4) — §4.1
- **4.2** Key Deliverables Catalog (D01 to D12) — §4.2
- **4.3** Risk Register & Proactive Mitigations (R01 to R07) — §4.3
- **4.4** Operational Assumptions & Architectural Constraints — §4.4

5 Work Breakdown Structure (WBS) for 6 Engineers — Section 5

- **5.1** Part A: 6-Engineer Task Ownership Profiles across 4 Weeks — §5.1
- **5.2** Part B: Week-by-Week Cross-Functional Sprints (Weeks 1 to 4) — §5.2

6 Responsibility Assignment Matrix (RACI) — Section 6

7 Appendix, Technical Glossary & Repository Standards — Section 7

SECTION 1

Executive Summary & Architecture

Overview of the Atlas AQAR methodology, core engineering principles, and measured baselines.

🏛️

PLATFORM ARCHITECTURE

LangGraph Multi-Agent

Stateful cyclic multi-agent orchestration coordinating research, discovery, math, and verification.

⏱️

PROJECT EXECUTION

4-Week Sprints

Structured 28-day agile plan executed across 6 specialized engineering roles.

🎯

NATIONAL INITIATIVE

DEPI Round 5

Ministry of Communications & IT — National Housing Authority Track NHA-5-072.

System Concept & Core Value Proposition

Atlas AQAR (أطلس عقار) is a bounded, production-grade multi-agent artificial intelligence system designed to conduct autonomous real-estate research, due-diligence, and investment analysis tailored specifically to the Egyptian property market. Rather than relying on simple, single-turn conversational chatbots that hallucinate figures and invent projects, Atlas AQAR implements a stateful, cyclic **LangGraph** graph workflow that breaks complex investment queries into structured sub-tasks, gathers grounded evidence via vector retrieval and developer verification, performs exact financial calculations, and passes all outputs through an automated editorial Critic agent.

A foundational design principle of Atlas AQAR is *deterministic financial computation*: all mortgage amortization, rental yields, capital growth projections, and sensitivity models are executed exclusively in dedicated mathematical algorithms, entirely preventing LLM arithmetic hallucinations. Every claim made in generated investment reports is directly tagged with an audit-ready Evidence ID (e.g., `[E001]`).

Empirical Performance Baseline

TASK COMPLETION RATE

100.0%

All 5 benchmark scenarios successfully fulfilled against target ≥80%.

CITATION PRECISION

100.0%

Zero uncited factual statements detected across verified evaluation runs.

FINANCIAL DETERMINISM

100.0%

Zero financial hallucination; mathematical outputs strictly match pure Python calculations.

CRITIC EVALUATION SCORE

100.0%

100% compliance across 7-point quality, evidence, and completeness criteria.

UNSUPPORTED CLAIM RATE

0.0%

Completely eliminated ungrounded assertions through evidence arsenal gating.

OFFLINE BENCHMARK LATENCY

0.12 s

High-speed evaluation pipeline executing 5 full research workflows.

Authoritative Project Execution Blueprint: This document represents the definitive implementation blueprint for Atlas AQAR. It outlines the complete four-week development methodology, stakeholder matrix, risk register, and the granular task allocation across the six specialized engineering roles who engineered the platform.

SECTION 2

Project Overview & Tech Stack

Market challenges, objectives, system boundaries, technology selections, and architectural foundations.

2.1 Background & Market Friction in Egyptian Real Estate

The Egyptian real-estate ecosystem is among the most dynamic investment landscapes in North Africa and the Middle East. However, individual and institutional investors face severe informational friction:

- **Data Fragmentation:** Unit listings, pricing indices, and developer track records are scattered across disparate websites, social groups, and informal brokers.
- **Information Asymmetry:** Potential buyers lack verified data on developer delivery track records, historical delays, and construction solvency.
- **Financial Calculation Opacity:** Real-estate installment plans often include hidden maintenance deposits, backloaded balloon payments, and fluctuating interest formulas that retail investors struggle to assess.
- **GenAI Unreliability:** Standard LLMs hallucinate property prices, invent developers, and introduce gross calculation errors when asked for real-estate ROI forecasts.

The Solution: Atlas AQAR provides an automated, evidence-grounded research co-pilot that combines semantic vector search, deterministic financial amortization, multi-agent cognitive critique, and professional report generation into a unified platform.

2.2 Strategic Vision, Mission & SMART Objectives

Vision: To become the leading trusted, verifiable AI intelligence engine for property research and real-estate investment decision-making in Egypt.

Mission: To democratize institutional-grade property due diligence by combining agentic reasoning with rigorous mathematical determinism and empirical evidence verification.

OBJECTIVE 1 (O1)

Autonomous Task Decomposition

Deconstruct user investment queries into 3–6 dependency-ordered sub-tasks executed systematically.

OBJECTIVE 2 (O2)

Deterministic Financial Engine

Deliver 100% accurate mortgage amortization, ROI, and sensitivity analysis via non-LLM Python algorithms.

OBJECTIVE 3 (O3)

Evidence-Grounded Reporting

Enforce mandatory citation of Evidence IDs (`[E001]`) for every quantitative claim and factual finding.

OBJECTIVE 4 (O4)

Automated Self-Critique & Refinement

Incorporate a dedicated Critic agent applying a 7-point evaluation rubric to guarantee report quality.

OBJECTIVE 5 (O5)

Multi-Model LLM Agnosticism

Support dynamic switching between OpenAI, Anthropic, Google Gemini, and fully offline mock LLMs.

OBJECTIVE 6 (O6)

Modular Production Architecture

Deploy via FastAPI backend, Streamlit interactive dashboard, and Docker Compose orchestration in 4 weeks.

2.3 Project Scope Boundaries & Non-Functional Exclusions

✓ FULLY IN SCOPE

- Multi-agent LangGraph workflow (Planner, Researcher, Analyzer, Writer, Critic).
- Curated vector store (ChromaDB) indexing property listings, developer profiles, and market reports.
- Deterministic financial engine (Installments, Yields, ROI, Scenarios).
- FastAPI REST service with versioned API endpoints (`/api/v1/research`, `/api/v1/financial`, etc.).
- Streamlit interactive web dashboard with 6 specialized analytical views.
- Automated evaluation suite with benchmark scenarios.
- Multi-container Docker Compose deployment.

✗ BOUNDARY EXCLUSIONS

- Real-time automated web scraping of dynamic listing portals without API licenses.
- User authentication, multi-tenant RBAC, and subscription billing gateways.
- Mobile native applications (iOS / Android).
- Direct integration with external banking / mortgage approval APIs.
- Multi-language report localization (Arabic report synthesis planned for v2.0).
- Automated legal contract drafting or conveyancing advice.

2.4 Architectural Stack & Layered Technology Matrix

ARCHITECTURAL TIER	PRIMARY TECHNOLOGY	VERSION	ROLE & ARCHITECTURAL RESPONSIBILITY
Agentic Orchestration	LangGraph & LangChain-Core	1.2.14 / 1.6.9	Stateful cyclic graph orchestration, agent execution, conditional edge routing.
Data Contracts & Validation	Pydantic v2	2.14.0	Strict typed data validation for GraphState, EvidenceArsenal, and analytical payloads.
Vector Retrieval & Embeddings	ChromaDB & HuggingFace	1.5.9 / 2.2.0	Persistent semantic document indexing, dense vector search, similarity ranking.
Financial Analytics	Pure Python Math Engine	3.11+	Deterministic mortgage amortization, net yields, capital appreciation, sensitivity grid.
Backend REST API	FastAPI & Uvicorn	0.143.0 / 0.54.0	Async REST service exposing workflow execution, health diagnostics, and OpenAPI specs.
Interactive Dashboard	Streamlit & Plotly	1.65.0 / 7.1.0	Analytical user interface, report rendering, interactive financial modeling graphs.
Testing & QA	pytest & pytest-asyncio	9.1.1 / 1.4.0	Automated unit testing, integration workflows, and benchmark assertions.
Container Infrastructure	Docker & Docker Compose	Latest	Multi-service containerization, environment reproducibility, volume persistence.

2.5 Empirical Performance Baseline & Acceptance Metrics

EVALUATION DIMENSION	EVALUATION METRIC	TARGET THRESHOLD	MEASURED BASELINE	AUDIT RESULT
Task Completion	Sub-task execution rate	≥ 80.0%	100.0%	PASSED

EVALUATION DIMENSION	EVALUATION METRIC	TARGET THRESHOLD	MEASURED BASELINE	AUDIT RESULT
Fact Verification	Citation Precision Score	> 90.0%	100.0%	PASSED
Financial Reliability	Zero-hallucination accuracy	100.0%	100.0%	PASSED
Editorial Quality	Critic 7-point pass rate	≥ 85.0%	100.0%	PASSED
Hallucination Gating	Unsupported claim frequency	≤ 2.0%	0.0%	PASSED
Evaluation Speed	Offline benchmark execution	< 5.0 s	0.12 s (5 cases)	PASSED

SECTION 3

Stakeholder Analysis

Identification, impact evaluation, power-interest classification, and communication management plan.

3.1 Internal Project Stakeholders

STAKEHOLDER GROUP	ORGANIZATIONAL ROLE	CORE PROJECT INTEREST	POWER	INTEREST	SUCCESS CRITERIA & EXPECTATIONS
Engineering Core (6 Engineers)	Full-Stack Implementation	Modular architecture, modern AI stack, clean code	High	High	Decoupled tasks, strict schemas, unblocked weekly sprint goals.
Project Lead / Scrum Master	Delivery & Alignment	On-time milestone delivery, sprint velocity	High	High	Transparency in progress, early risk escalation, milestone governance.
AI / ML Architect	Technical Direction	System resilience, determinism, agentic bounds	High	High	Strict adherence to Pydantic contracts and LangGraph cyclic patterns.
QA & Verification Lead	Quality Assurance	Defect elimination, benchmark repeatability	Medium	High	Comprehensive test coverage across edge cases, reproducible benchmarks.
DevOps & Infra Lead	Deployment Operations	Container stability, zero configuration drift	Medium	Medium	Hermetic Docker Compose builds, volume persistence, isolated environments.
Technical Documentation Lead	Knowledge Management	Accurate publications, professional reports	Low	Medium	Systematic documentation, clear API specs, presentation-ready artifacts.

3.2 External Ecosystem Stakeholders

STAKEHOLDER ENTITY	CATEGORY	INSTITUTIONAL INTEREST	POWER	INTEREST	PRIMARY ENGAGEMENT OBJECTIVE
DEPI Leadership & Mentors	Governance / Supervisory	High-caliber national AI graduation showcase	High	High	Deliver compliant, demonstrable AI product meeting all program standards.
National Housing Authority (NHA)	Institutional Beneficiary	Data-driven real-estate analytics tool	High	High	Provide verifiable, audit-proof housing due-diligence capabilities.

STAKEHOLDER ENTITY	CATEGORY	INSTITUTIONAL INTEREST	POWER	INTEREST	PRIMARY ENGAGEMENT OBJECTIVE
Real-Estate Investors (End Users)	Target Consumer Base	Actionable, grounded, trustworthy reports	Medium	High	Eliminate investment uncertainty with transparent financial schedules.
Property Developers	Market Ecosystem	Accurate portrayal of project timelines and specs	Medium	Medium	Maintain factual data accuracy regarding pricing and payment terms.
Academic & Peer Reviewers	Advisory / Academic	Technical novelty, code rigor, repeatability	Low	Medium	Publish clear architectural documentation and benchmark proofs.
LLM Infrastructure Providers	External Technology Vendor	API consumption compliance and quota usage	Medium	Low	Maintain bounded token budgets (100k cap) and offline fallback modes.

3.3 Power / Interest 2×2 Matrix

MONITOR (LOW INTEREST, HIGH POWER)

LLM Infrastructure Providers, Regulatory Bodies

Ensure API stability and compliance; track usage quotas and maintain offline fallback modes.

MANAGE CLOSELY (HIGH INTEREST, HIGH POWER)

DEPI Leadership, NHA Stakeholders, Project Lead, AI Architect

Primary project drivers. Require weekly milestone updates, prototype demos, and formal reviews.

KEEP INFORMED (LOW INTEREST, LOW POWER)

Academic Mentors, Technical Documentation Team

Keep apprised of project progress, architectural ADRs, and documentation snapshots.

KEEP SATISFIED (HIGH INTEREST, MEDIUM/LOW POWER)

Real-Estate Investors, Engineering Team, QA & DevOps Leads

Ensure user-friendly UI, deterministic accuracy, sprint satisfaction, and unblocked tasks.

3.4 Comprehensive Stakeholder Communication Plan

AUDIENCE	COMMUNICATION CHANNEL	FREQUENCY	ARTIFACT / CONTENT	RESPONSIBLE OWNER
Engineering Team	GitHub Issues & Stand-up	Daily	Task burndown, blocker resolution, PR reviews	Project Lead
DEPI Program Leadership	Formal Progress Briefing	Weekly (Fridays)	Milestone status, risk register updates, deliverables	Project Lead / AI Architect
NHA Institutional Partners	Live Product Demonstration	Bi-weekly	Working system demo, feedback logs, compliance check	Project Lead

AUDIENCE	COMMUNICATION CHANNEL	FREQUENCY	ARTIFACT / CONTENT	RESPONSIBLE OWNER
Peer Reviewers & Mentors	Documentation Drive & GitHub	Per Milestone	Architecture diagrams, benchmark tables, code reviews	Technical Documentation Lead
End Users / Investors	Interactive Dashboard Tour	Release Milestone	User Guide, quick-start videos, sample reports	QA / Product Lead

3.5 Strategic Engagement & Escalation Protocols

GOVERNANCE ENGAGEMENT PROTOCOL

All formal project submissions to DEPI leadership are accompanied by this Project Execution Plan, reproducible test scripts, and an interactive demonstration. Any architectural deviations from the original proposal require prior review.

TECHNICAL COMMUNITY PROTOCOL

Open communication via structured GitHub PR templates. Every PR must pass automated unit tests (`pytest tests/unit/`) and linting before merge approval. Weekly retrospective meetings capture lessons learned.

SECTION 4

Project Plan, Milestones & Risks

Milestone roadmap, deliverables catalog, comprehensive risk management, and operational boundaries.

4.1 Chronological Project Milestones (Sprint 1 to Sprint 4)

WEEK 1 · DAYS 1–7

Milestone 1 (M1) — Environment & Data Foundation Baseline

Repository established at `nhahub/NHA-5-072`. Pydantic v2 domain schemas finalized. Raw property and developer datasets curated and indexed in ChromaDB vector database. System configuration layer and benchmark harness operational.

WEEK 2 · DAYS 8–14

Milestone 2 (M2) — Agent Suite & Deterministic Engine Core

Implementation of 5 LangGraph agent nodes (Planner, Researcher, Analyzer, Writer, Critic). Full 8-tool registry active. Deterministic financial amortization engine completed and validated with 100% test coverage. Cognitive cyclic loop verified.

WEEK 3 · DAYS 15–21

Milestone 3 (M3) — API Services & Interactive Web Dashboard

FastAPI REST service operational with `/api/v1/` routes. Streamlit frontend completed featuring 6 analytical panels (Research Console, Verified Report Viewer, Property Grid, Financial Calculator, Evidence Inspector, and Observability Monitor).

WEEK 4 · DAYS 22–28

Milestone 4 (M4) — Containerization, Benchmarks & Final Release

Multi-service Docker Compose deployment validated. Formal evaluation benchmarks executed and performance metrics cataloged. Technical documentation published. Final Git tag pushed and submitted.

4.2 Key Deliverables Catalog

ID	DELIVERABLE NAME	CATEGORY	TARGET WEEK	DELIVERY STATUS
D01	Official GitHub Repository (nhahub/NHA-5-072)	Source Code	Week 1	Completed
D02	Pydantic v2 Core Data Models & GraphState	Architecture	Week 1	Completed
D03	ChromaDB Vector Store & Ingestion Pipeline	Data Infra	Week 1	Completed
D04	5-Agent LangGraph Cognitive Workflow	AI Intelligence	Week 2	Completed
D05	8-Tool Specialized Domain Suite	Tools / Engine	Week 2	Completed
D06	Deterministic Financial Calculation Engine	Analytics Engine	Week 2	Completed
D07	FastAPI REST API Service (v1 Endpoints)	Backend Service	Week 3	Completed
D08	Streamlit Multi-Panel Interactive Dashboard	User Interface	Week 3	Completed
D09	Multi-Container Docker Compose Deployment	DevOps	Week 4	Completed
D10	Empirical Evaluation & Benchmark Report	Quality / QA	Week 4	Completed
D11	Comprehensive Project Plan & WBS PDF	Documentation	Week 4	Completed

ID	DELIVERABLE NAME	CATEGORY	TARGET WEEK	DELIVERY STATUS
D12	Final Project Submission & Presentation Package	Release	Week 4	Completed

4.3 Risk Register & Proactive Mitigations

ID	RISK DESCRIPTION	LIKELIHOOD	IMPACT	SEVERITY	PROACTIVE MITIGATION STRATEGY
R01	External LLM API quota exhaustion / rate limiting	Medium	High	High	Implement automatic offline fallback mock provider; enforce 100k session token budget; support multiple provider API keys.
R02	Upstream dependency breaking changes (LangGraph / ChromaDB)	Low	High	High	Pin exact library versions in `requirements.txt`; isolate dependencies in hermetic virtual environments.
R03	Vector database persistence corruption or re-indexing lag	Low	Medium	Medium	Write idempotent ingestion scripts; persist ChromaDB to dedicated mounted volumes; add health check probe on startup.
R04	Edge-case financial calculation discrepancies (balloon payments)	Medium	High	Medium	Implement 100% unit test coverage for mathematical engine; enforce strict Pydantic numeric validation; isolate math from LLM prompts.
R05	Inter-workstream integration delays during Week 3	Medium	Medium	Medium	Define typed Pydantic interfaces in Week 1; use mock responses for parallel backend and frontend development.
R06	OS environment discrepancies (Windows PowerShell vs Linux Docker)	Medium	Low	Low	Test Docker Compose builds locally across platforms; provide explicit `.env.example` with platform-independent path resolution.
R07	Streamlit state loss or excessive re-rendering latency	Low	Low	Low	Use `st.session_state` caching for immutable data; paginate property tables; lazy-load complex report graphs.

4.4 Operational Assumptions & Architectural Constraints

CORE PROJECT ASSUMPTIONS

- All 6 engineers operate on Python 3.11+ with local Docker Desktop installed.
- Curated JSON datasets accurately reflect representative Egyptian real-estate market segments in New Cairo, New Capital, and Sheikh Zayed.
- At least one live API provider key (OpenAI / Gemini / Anthropic) is accessible for validation, while offline mock mode is available for local testing.

OPERATIONAL CONSTRAINTS

- Maximum of 10 LangGraph workflow iterations per research request to prevent infinite loops.
- Maximum of 3 Critic revision retries before forced report synthesis.
- Zero arithmetic calculations delegated to LLM text generation.
- System must be fully runnable locally without mandatory paid cloud dependencies.

- Four-week timeline is firm with fixed weekly sprint reviews.

SECTION 5

Work Breakdown Structure (WBS) for 6 Engineers

Granular task distribution across the four-week sprint, clearly apportioned among six dedicated, specialized engineering roles to demonstrate end-to-end team ownership and operational methodology.

5.1 Part A: 6-Engineer Task Ownership Profiles across 4 Weeks

Engineer 1 — AI & Multi-Agent Systems Architect

LangGraph Orchestration, State Graph, Flow Controls

Core Domain: Master architecture, state schema validation, cyclic graph compilation, conditional edge routing, iteration safeguards, and OpenTelemetry instrumentation.

WEEK 1 · SPRINT 1

- Architect LangGraph StateGraph topology & cyclic transitions.
- Define `GraphState` & `EvidenceArsenal` Pydantic v2 models.
- Establish runtime iteration caps (10 max) & token budget rules.

WEEK 2 · SPRINT 2

- Compile `src/graph/workflow.py` connecting 5 agent nodes.
- Implement conditional `src/graph/edges.py` . edge routing in
- Wire error handling & retry handlers for cycle resilience.

WEEK 3 · SPRINT 3

- Optimize multi-model provider adapter (OpenAI / Gemini / Offline).
- Instrument OpenTelemetry hooks for tracing and latency profiling.
- Audit state immutability to prevent memory leaks during long runs.

Engineer 2 — Cognitive Reasoning & LLM Agent Specialist

Prompt Engineering, ReAct Cognitive Loops, Verification Rubrics

Core Domain: Agent persona formulation, structured reasoning prompts, ReAct execution cycles, grounded report synthesis, and Critic editorial gating.

WEEK 1 · SPRINT 1

- Author `planner.yaml` prompt for query decomposition.
- Build Planner logic generating 3–6 dependency-ordered sub-tasks.
- Validate sub-task JSON schema output against Pydantic models.

WEEK 2 · SPRINT 2

- Implement Researcher ReAct loop (Thought-Action-Observation).
- Build Writer agent synthesizing

WEEK 3 · SPRINT 3

- Fine-tune `[E001]` citation Writer prompts for strict inline placement.
- Refine Critic feedback format for targeted revision directives.

WEEK 4 · SPRINT 4

- Run `scripts/run_e` evaluation harness
- Verify 100% Citation Preci assertions.
- Capture representative rep presentation artifacts.

9-section structured reports.
- Build Critic agent applying 7-point factual and math rubric.

- Verify 100% evidence chain traceability from report to raw facts.

Engineer 3 — Data Engineering & RAG Knowledge Base Specialist

ChromaDB Vector Store, Semantic Embeddings, Dataset Curation

Core Domain: Curating real-estate listing corpora, developer solvency histories, market intelligence reports, vector database indexing, and semantic retrieval tools.

WEEK 1 · SPRINT 1

- Curate `properties.json` covering New Cairo, Capital, Zayed.
- Compile `developers.json` with delivery track record scores.
- Build `scripts/ingest_data.py` . ChromaDB ingestion script in

WEEK 2 · SPRINT 2

- Implement `RAGRetrieverTool` querying ChromaDB with top-k scoring.
- Implement `PropertySearchTool` & `PropertyDetailsTool` .
- Implement `DeveloperLookupTool` extracting historical delivery data.

WEEK 3 · SPRINT

- Ingest supplemental market reports in
- Implement `WebS`
- Tune embedding optimal semantic

Engineer 4 — Quantitative & Deterministic Financial Algorithms Specialist

Mortgage Amortization, Yield Modeling, ROI & Sensitivity Analysis

Core Domain: Non-LLM mathematical algorithms, installment schedule calculation, delivery balloon modeling, compound capital growth, net rental yields, and sensitivity matrices.

WEEK 1 · SPRINT 1

- Specify formulas for monthly amortization & balloon payments.
- Implement `src/engine/installments.py` . core algorithms in
- Write unit test assertions for exact 100% price sum match.

WEEK 2 · SPRINT 2

- Build capital appreciation & rental yield logic `roi.py` . in
- Build sensitivity scenario `src/engine/scenarios.py` . analyzer in
- Package `InstallmentCalcTool` , `ROIEstimatorTool` . tools:

WEEK 3 · SPRINT 3

- Add inflation & rental demand fluctuation multipliers to scenarios.
- Implement EGP / USD currency formatters

& numeric parsers.
- Validate financial serialization against Pydantic DTO schemas.

Engineer 5 — Backend Systems & REST API Integration Specialist

FastAPI Services, REST Endpoints, Async Workers, Observability

Core Domain: FastAPI application lifecycle, REST route handlers, request validation, async execution of LangGraph workflows, health probes, and OpenAPI documentation.

WEEK 1 · SPRINT 1

- Scaffold FastAPI application structure in `src/api/main.py`.
- Define request/response DTO schemas in `src/api/schemas.py`.
- Implement health check endpoint `GET /api/v1/health`.

WEEK 2 · SPRINT 2

- Implement `POST /api/v1/research` executing the LangGraph graph.
- Implement `GET /api/v1/properties` with multi-filter query params.
- Implement `POST /api/v1/financial` for standalone math runs.

WEEK 3 · SPRINT 3

- Implement error handling middlewares returning JSON error envelopes.
- Generate and audit comprehensive OpenAPI docs at `/docs` interactive.
- Implement rate limiting and query timeout guards for API stability.

Engineer 6 — Frontend UI, QA & DevOps Infrastructure Specialist

Streamlit Dashboard, Plotly Analytics UI, Docker Compose, CI/CD

Core Domain: Streamlit interactive dashboard, Plotly analytical charts, comprehensive pytest suites, multi-container Docker Compose setup, and official PDF publication.

WEEK 1 · SPRINT 1

- Init GitHub repo at `nhahub/NHA-5-072` & branch rules.
- Configure `pyproject.toml`, virtualenv, and requirements.
- Draft baseline Dockerfile and docker-compose.yml services.

WEEK 2 · SPRINT 2

- Write unit test suites for tools and financial calculation engine.
- Scaffold Streamlit layout in `src/ui/dashboard.py`.
- Apply custom CSS design tokens and theme color palettes.

WEEK 3 · SPRINT 3

- Complete all 6 Streamlit UI panels (Console, Report, Calculator).
- Implement Plotly visualizations (scatter, amortization bars, ROI gauge).
- Author full integration test in `test_workflow.py` (100% pass).

5.2 Part B: Week-by-Week Cross-Functional Sprints (Weeks 1 to 4)

Week 1 — Sprint 1: Architecture, Data Ingestion & Core Modeling — Days 1 to 7

Engineer 1 AI ARCHITECT	▸ Architect LangGraph StateGraph topology, cyclic schema & 10 Pydantic v2 data models.
Engineer 2 LLM SPECIALIST	▸ Author `planner.yaml` prompt template & implement Planner task decomposition logic.
Engineer 3 DATA SPECIALIST	▸ Curate `properties.json` & `developers.json` ; build ChromaDB ingestion pipeline.
Engineer 4 FINANCIAL SPECIALIST	▸ Implement core mortgage amortization formulas in `src/engine/installments.py` with unit tests.
Engineer 5 BACKEND SPECIALIST	▸ Scaffold FastAPI service in `src/api/main.py` , DTO schemas & `GET /health` endpoint.
Engineer 6 DEVOPS / UI	▸ Init GitHub repo at `nhahub/NHA-5-072` , setup virtualenv, dependencies & Dockerfile skeleton.

Week 2 — Sprint 2: Multi-Agent Logic, Tool Registry & Financial Engine — Days 8 to 14

Engineer 1 AI ARCHITECT	▸ Compile `src/graph/workflow.py` connecting 5 agents; wire conditional edge routing.
Engineer 2 LLM SPECIALIST	▸ Build Researcher ReAct loop, 9-section Writer synthesis & 7-point Critic evaluation rubric.
Engineer 3 DATA SPECIALIST	▸ Build `RAGRetrieverTool` , `PropertySearchTool` , `DeveloperLookupTool` . and
Engineer 4 FINANCIAL SPECIALIST	▸ Build ROI & rental yield engine in `roi.py` and sensitivity analyzer in `scenarios.py` .
Engineer 5 BACKEND SPECIALIST	▸ Implement `POST /research` , `GET /properties` , and `POST /financial` endpoints.
Engineer 6 DEVOPS / UI	▸ Write unit tests in `test_tools.py` & scaffold Streamlit layout in `src/ui/dashboard.py` .

Week 3 — Sprint 3: Full-Stack Integration, Web Dashboard & End-to-End QA — Days 15 to 21

Engineer 1 AI ARCHITECT	▸ Optimize multi-model provider adapter; instrument OpenTelemetry hooks for tracing.
Engineer 2 LLM SPECIALIST	▸ Fine-tune Writer prompts for strict `[E001]` citation placement & verify evidence chain integrity.
Engineer 3 DATA SPECIALIST	▸ Ingest `market_reports.json`; implement `WebSearchTool` fallback with HTML sanitization.
Engineer 4 FINANCIAL SPECIALIST	▸ Add inflation & demand multipliers to scenario models; implement EGP/USD currency formatters.
Engineer 5 BACKEND SPECIALIST	▸ Finalize error handling middlewares; audit OpenAPI Swagger docs at `/docs`; add rate limiting.
Engineer 6 DEVOPS / UI	▸ Complete all 6 Streamlit UI panels with Plotly charts; run integration test suite (100% pass).

Week 4 **Sprint 4: Benchmarking, Containerization, Release & Presentation**	Days 22 to 28
Engineer 1 AI ARCHITECT	▸ Perform architectural code audit across graph and tool boundaries; record ADR documents.
Engineer 2 LLM SPECIALIST	▸ Execute benchmark suite via `scripts/run_evaluation.py`; verify 100% Citation Precision.
Engineer 3 DATA SPECIALIST	▸ Lock production dataset snapshots with SHA-256; verify ChromaDB volume persistence.
Engineer 4 FINANCIAL SPECIALIST	▸ Perform boundary tests (0% down payments); verify 100% zero-hallucination compliance.
Engineer 5 BACKEND SPECIALIST	▸ Execute concurrency stress test (10 simultaneous queries); publish complete API Reference.
Engineer 6 DEVOPS / UI	▸ Validate `docker-compose up --build`; generate final PDF plan; push release tag to GitHub.

SECTION 6

Responsibility Assignment Matrix (RACI)

Clear accountability mapping of the six engineers across all primary project deliverables (R = Responsible, A = Accountable, C = Consulted, I = Informed).

DELIVERABLE ID & NAME	ENG 1 (ARCHITECT)	ENG 2 (LLM/AGENT)	ENG 3 (DATA/RAG)	ENG 4 (FINANCE)	ENG 5 (BACKEND)	ENG 6 (UI/DEVOPS)
D01: GitHub Repository & CI Skeleton	C	I	I	I	C	**R / A**
D02: Pydantic Data Models & GraphState	**R / A**	C	C	C	C	I
D03: ChromaDB Vector Store & Ingestion	C	C	**R / A**	I	I	C
D04: 5-Agent LangGraph Cognitive Flow	**R / A**	**R**	C	C	C	I
D05: 8-Tool Specialized Domain Suite	C	C	**R**	**R**	C	C
D06: Deterministic Financial Engine	C	I	I	**R / A**	C	C
D07: FastAPI REST Service (v1 Endpoints)	C	I	I	C	**R / A**	C
D08: Streamlit Interactive UI (6 Panels)	I	C	I	C	C	**R / A**
D09: Multi-Container Docker Compose	C	I	C	I	C	**R / A**
D10: Evaluation & Benchmark Metrics	C	**R / A**	C	C	I	C
D11: Project Execution Plan PDF	C	C	C	C	C	**R / A**

DELIVERABLE ID & NAME	ENG 1 (ARCHITECT)	ENG 2 (LLM/AGENT)	ENG 3 (DATA/RAG)	ENG 4 (FINANCE)	ENG 5 (BACKEND)	ENG 6 (UI/DEVOPS)
D12: Final Project Submission & Review	A	R	R	R	R	R

Governance Guarantee: Every deliverable has exactly one primary Accountable (A) engineer, with complementary Responsible (R) contributors, completely eliminating ambiguity and ensuring strict operational throughput across the 4-week execution cycle.

SECTION 7

Appendix, Technical Glossary & Standards

Terminology definitions, abbreviation index, and official reference links.

Glossary of Key Technical Terms

TERM	TECHNICAL DEFINITION WITHIN ATLAS AQAR
Agentic Workflow	An AI design pattern where an LLM coordinates reasoning, tool invocation, observation, and state updates iteratively rather than in a single-prompt response.
LangGraph	A specialized framework built on LangChain for building stateful, multi-actor applications with cyclic graph computation and persistence.
GraphState	The centralized, immutable Pydantic v2 data model passed between all agents in the workflow, tracking queries, sub-tasks, evidence records, drafts, and critique notes.
Evidence Arsenal	A structured repository of verified facts, listing specifications, and financial calculations collected during the research phase, each carrying a unique Evidence ID.
Evidence ID	A deterministic citation tag (e.g., `[E001]`, `[E002]`) assigned to raw data items and referenced inline throughout generated analytical reports.
Deterministic Financial Engine	A pure Python mathematical module executing mortgage amortization, ROI, and sensitivity analysis arithmetically, completely bypassing LLM probabilistic text generation.
ReAct Loop	A cognitive pattern (Reasoning + Acting) where the Researcher agent generates thoughts, executes tool actions, observes results, and loops until sufficient evidence is accumulated.
Critic Agent	An editorial inspection agent that evaluates draft reports against a 7-point rubric (factual grounding, math accuracy, completeness) and issues conditional approval or revision feedback.
ChromaDB	An open-source, embedded vector database used in Atlas AQAR to store and semantically retrieve high-dimensional embeddings of properties and market reports.
DEPI	Digital Egypt Pioneers Initiative — the national training and incubation initiative under which Atlas AQAR is developed within Round 5.

Official Project Reference Links

- **GitHub Repository:** https://github.com/nhahub/NHA-5-072
- **LangGraph Framework:** https://langchain-ai.github.io/langgraph/
- **FastAPI Documentation:** https://fastapi.tiangolo.com/
- **ChromaDB Vector Store:** https://docs.trychroma.com/
- **Pydantic v2 Documentation:** https://docs.pydantic.dev/latest/
- **Streamlit Platform:** https://docs.streamlit.io/